

COMP3026E Distributed Systems

14. Databases

James Jianqiao Yu

Harbin Institute of Technology, Shenzhen

- Background on databases
- More on isolation
 - MVCC and snapshot isolation
- More on sharding
 - Data distribution strategies



Key-value Stores vs. Relational Databases



- Key-value stores
 - Get(K)
 - Put(K, V)
 - Delete(K)

Key	Value
as1234	{"name": "Alice Smith", "major": "CSE"}
bw5678	{"name": "Bob Williams", "major": "ECE"}



Key-value Stores vs. Relational Databases



- Relational databases
 - Tables (i.e., “relations”) store records
 - Can select/filter records by different columns
 - Tables can be joined
 - Fixed schema

netid	first_name	last_name	major
as1234	Alice	Smith	CSE
bw5678	Bob	Williams	ECE

major	enrollment
CSE	400
ECE	300



MVCC: Multi-Version Concurrency Control



- Store multiple versions of each record

netid	first_name	last_name	major	version
as1234	Alice	Smith	CSE	0
bw5678	Bob	Williams	ECE	0



MVCC: Multi-Version Concurrency Control



- Store multiple versions of each record

netid	first_name	last_name	major	version
as1234	Alice	Smith	CSE	0
as1234	Alice	Smith	ECO	1
bw5678	Bob	Williams	ECE	0



MVCC: Multi-Version Concurrency Control



- Store multiple versions of each record
 - Allows “time travel”
 - Enables snapshot isolation

netid	first_name	last_name	major	start_ts	end_ts
as1234	Alice	Smith	CSE	Sep 2024	Oct 2025
as1234	Alice	Smith	ECO	Oct 2025	Present
bw5678	Bob	Williams	ECE	Sep 2025	Present

MVCC: Multi-Version Concurrency Control

- Store multiple versions of each record
 - Allows “time travel”
 - Enables snapshot isolation

netid	first_name	last_name	major	start_xid	end_xid
as1234	Alice	Smith	CSE	100	200
as1234	Alice	Smith	ECO	200	inf
bw5678	Bob	Williams	ECE	150	inf

Snapshot Isolation



- A transaction reads from a snapshot taken at start time
- No locks for reads!
- Conflicts may arise due to writes
 - If two transactions write the same record, the first transaction commits and the second transaction aborts (write-write conflict)
 - Does **not** check for read-write conflicts

Snapshot Isolation

- A transaction reads from a snapshot taken at start time
- No locks for reads!
- Conflicts may arise due to writes
 - If two transactions write the same record, the first transaction commits and the second transaction aborts (write-write conflict)
 - Does **not** check for read-write conflicts

T1:	R(A)->10	R(A)->20
T2:	W(A=20)	

Snapshot Isolation

- A transaction reads from a snapshot taken at start time
- No locks for reads!
- Conflicts may arise due to writes
 - If two transactions write the same record, the first transaction commits and the second transaction aborts (write-write conflict)
 - Does **not** check for read-write conflicts

A=100

A=90

A=90

B=100

B=100

B=110

Transfer: R(A) W(A)

R(B) W(B)

Sum:

R(A) R(B)

Snapshot Isolation in Postgres

- Each transaction is committed with a monotonically increasing transaction ID (`xid`)
 - New records created by transaction will have `xmin = xid`
 - Records deleted by transaction will have `xmax = xid`

Snapshot Isolation in Postgres



key	value	xmin	xmax
A	10	0	inf

T1 (xid=1)

W(A, 20)

Snapshot Isolation in Postgres

key	value	xmin	xmax
A	10	0	1
A	20	1	inf

T1 (xid=1)

W(A, 20)

Snapshot Isolation in Postgres

- Each transaction is committed with a monotonically increasing transaction ID (`xid`)
 - New records created by transaction will have `xmin = xid`
 - Records deleted by transaction will have `xmax = xid`
- Snapshot is defined by three variables
 - `xmin`: oldest in-progress transaction ID at the time the snapshot was taken
 - `xmax`: next available transaction ID (not taken by committed or in-progress transactions)
 - `xip`: list of concurrent in-progress transaction IDs

Snapshot Isolation in Postgres

- Each transaction is committed with a monotonically increasing transaction ID (`xid`)
 - New records created by transaction will have `xmin = xid`
 - Records deleted by transaction will have `xmax = xid`
- Visibility check for a given record
 - Record must be created
 - `xmin < snapshot xmin`, OR
 - `snapshot xmin ≤ xmin < snapshot xmax` AND `xmin` not in `xip`
 - Record must not be deleted
 - `xmax ≥ snapshot xmax` OR `xmax` in `xip`

Snapshot Isolation in Postgres

key	value	xmin	xmax
A	10	0	inf

nextXID = 1

T1

xmin = 1

xmax = 1

xip = {}

Snapshot Isolation in Postgres

key	value	xmin	xmax
A	10	0	inf

- Record must be created
 - $xmin < \text{snapshot } xmin$, OR
 - $\text{snapshot } xmin \leq xmin < \text{snapshot } xmax$ AND $xmin$ not in xip
- Record must not be deleted
 - $xmax \geq \text{snapshot } xmax$ OR $xmax$ in xip

nextXID = 1

T1

$xmin = 1$

$xmax = 1$

$xip = \{\}$

$R(A) \rightarrow ?$

Snapshot Isolation in Postgres

key	value	xmin	xmax
A	10	0	inf

- Record must be created
 - `xmin < snapshot xmin`, OR
 - `snapshot xmin <= xmin < snapshot xmax` AND `xmin not in xip`
- Record must not be deleted
 - `xmax >= snapshot xmax` OR `xmax in xip`

nextXID = 1

T1

`xmin = 1`

`xmax = 1`

`xip = {}`

`R(A) -> 10`

Snapshot Isolation in Postgres

key	value	xmin	xmax
A	10	0	inf

- Record must be created
 - $xmin < \text{snapshot } xmin$, OR
 - $\text{snapshot } xmin \leq xmin < \text{snapshot } xmax$ AND $xmin$ not in xip
- Record must not be deleted
 - $xmax \geq \text{snapshot } xmax$ OR $xmax$ in xip

nextXID = 2

T1(xid=1)

$xmin = 1$

$xmax = 1$

$xip = \{\}$

$R(A) \rightarrow 10$

$W(A, 20)$

Snapshot Isolation in Postgres

key	value	xmin	xmax
A	10	0	1
A	20	1	inf

- Record must be created
 - $xmin < \text{snapshot } xmin$, OR
 - $\text{snapshot } xmin \leq xmin < \text{snapshot } xmax$ AND $xmin$ not in xip
- Record must not be deleted
 - $xmax \geq \text{snapshot } xmax$ OR $xmax$ in xip

nextXID = 2

T1(xid=1)

xmin = 1

xmax = 1

xip = {}

R(A) -> 10

W(A, 20)

Snapshot Isolation in Postgres

key	value	xmin	xmax
A	10	0	1
A	20	1	inf

- Record must be created
 - $xmin < snapshot\ xmin$, OR
 - $snapshot\ xmin \leq xmin < snapshot\ xmax$ AND $xmin$ not in xip
- Record must not be deleted
 - $xmax \geq snapshot\ xmax$ OR $xmax$ in xip

nextXID = 2

T1(xid=1)

xmin = 1

xmax = 1

xip = {}

R(A) -> 10

W(A, 20)

T2

xmin = 1

xmax = 2

xip = {1}

R(A) -> ?

Snapshot Isolation in Postgres

key	value	xmin	xmax
A	10	0	1
A	20	1	inf

- Record must be created
 - $xmin < \text{snapshot } xmin$, OR
 - $\text{snapshot } xmin \leq xmin < \text{snapshot } xmax$ AND $xmin$ not in xip
- Record must not be deleted
 - $xmax \geq \text{snapshot } xmax$ OR $xmax$ in xip

nextXID = 2

T1(xid=1)

xmin = 1

xmax = 1

xip = {}

R(A) -> 10

W(A, 20)

T2

xmin = 1

xmax = 2

xip = {1}

R(A) -> 10

Snapshot Isolation in Postgres

key	value	xmin	xmax
A	10	0	1
A	20	1	inf
B	30	2	inf

- Record must be created
 - $xmin < \text{snapshot } xmin$, OR
 - $\text{snapshot } xmin \leq xmin < \text{snapshot } xmax$ AND $xmin$ not in xip
- Record must not be deleted
 - $xmax \geq \text{snapshot } xmax$ OR $xmax$ in xip

nextXID = 3

T1(xid=1)

xmin = 1

xmax = 1

xip = {}

R(A) -> 10

W(A, 20)

T2(xid=2)

xmin = 1

xmax = 2

xip = {1}

R(A) -> 10

W(B, 30)

commit

Snapshot Isolation in Postgres

key	value	xmin	xmax
A	10	0	1
A	20	1	inf
B	30	2	inf

- Record must be created
 - $xmin < \text{snapshot } xmin$, OR
 - $\text{snapshot } xmin \leq xmin < \text{snapshot } xmax$ AND $xmin$ not in xip
- Record must not be deleted
 - $xmax \geq \text{snapshot } xmax$ OR $xmax$ in xip

nextXID = 3

T1(xid=1)

xmin = 1

xmax = 1

xip = {}

R(A) -> 10

W(A, 20)

T3

xmin = 1

xmax = 3

xip = {1}

D(B)

Snapshot Isolation in Postgres

key	value	xmin	xmax
A	10	0	1
A	20	1	inf
B	30	2	3

- Record must be created
 - $xmin < \text{snapshot } xmin$, OR
 - $\text{snapshot } xmin \leq xmin < \text{snapshot } xmax$ AND $xmin$ not in xip
- Record must not be deleted
 - $xmax \geq \text{snapshot } xmax$ OR $xmax$ in xip

nextXID = 4

T1(xid=1)

$xmin = 1$

$xmax = 1$

$xip = \{\}$

R(A) $\rightarrow$ 10

W(A, 20)

T3(xid=3)

$xmin = 1$

$xmax = 3$

$xip = \{1\}$

D(B)

Snapshot Isolation in Postgres

key	value	xmin	xmax
A	10	0	1
A	20	1	inf
B	30	2	3

- Record must be created
 - $xmin < \text{snapshot } xmin$, OR
 - $\text{snapshot } xmin \leq xmin < \text{snapshot } xmax$ AND $xmin$ not in xip
- Record must not be deleted
 - $xmax \geq \text{snapshot } xmax$ OR $xmax$ in xip

nextXID = 4

T1(xid=1)

$xmin = 1$

$xmax = 1$

$xip = \{\}$

R(A) $\rightarrow$ 10

W(A, 20)

T3(xid=3)

$xmin = 1$

$xmax = 3$

$xip = \{1\}$

D(B)

R(A) $\rightarrow$ 10

Snapshot Isolation in Postgres

key	value	xmin	xmax
A	10	0	1
A	20	1	inf
B	30	2	3

- Record must be created
 - $xmin < \text{snapshot } xmin$, OR
 - $\text{snapshot } xmin \leq xmin < \text{snapshot } xmax$ AND $xmin$ not in xip
- Record must not be deleted
 - $xmax \geq \text{snapshot } xmax$ OR $xmax$ in xip

nextXID = 4

T1(xid=1)

$xmin = 1$

$xmax = 1$

$xip = \{\}$

R(A) $\rightarrow$ 10

W(A, 20)

T3(xid=3)

$xmin = 1$

$xmax = 3$

$xip = \{1\}$

D(B)

R(A) $\rightarrow$ 10

W(A, 40)



Snapshot Isolation vs. Serializability



- Serializability is a stronger isolation level
 - Snapshot isolation allows schedules that are not serializable (next slide)
- Snapshot isolation is more scalable
 - Reads and writes do not block each other, unlike 2PL

Write Skew

- Occurs when transactions **read overlapping sets of data** but **write disjoint sets**
 - Snapshot isolation only checks for write-write conflicts on the same record

key	value
A	1
B	0

T1

(turn all 1s to 0s)

R(A)

R(B)

W(A, 0)

T2

(turn all 0s to 1s)

R(A)

R(B)

W(B, 1)

Write Skew

- Occurs when transactions **read overlapping sets of data** but **write disjoint sets**
 - Snapshot isolation only checks for write-write conflicts on the same record

key	value
A	1
B	0

T1

(turn all 1s to 0s)

R(A)

R(B)

W(A, 0)

T2

(turn all 0s to 1s)

R(A)

R(B)

W(B, 1)

key	value
A	0
B	1

Not serializable!



Isolation vs. Consistency



- Isolation is about whether concurrent transactions interfere with each other
 - Isolation models: serializable, snapshot isolation
- Consistency is about whether nodes see the same state
 - Consistency models: linearizable, causal+, eventual

- Background on databases
- More on isolation
 - MVCC and snapshot isolation
- More on sharding
 - Data distribution strategies



Data Distribution Strategies



- Example: distribute data across three nodes

	Node 1	Node 2	Node 3
Range	Netid in [aa0000, hz9999]	Netid in [ia0000, sz0000]	Netid in [ta0000, zz9999]
Round robin			
Hash			
All / Broadcast			



Data Distribution Strategies



- Example: distribute data across three nodes

	Node 1	Node 2	Node 3
Range	Netid in [aa0000, hz9999]	Netid in [ia0000, sz0000]	Netid in [ta0000, zz9999]
Round robin	Row 1, row 4, row 7, ...	Row 2, row 5, row 8, ...	Row 3, row 6, row 9, ...
Hash			
All / Broadcast			



Data Distribution Strategies



- Example: distribute data across three nodes

	Node 1	Node 2	Node 3
Range	Netid in [aa0000, hz9999]	Netid in [ia0000, sz0000]	Netid in [ta0000, zz9999]
Round robin	Row 1, row 4, row 7, ...	Row 2, row 5, row 8, ...	Row 3, row 6, row 9, ...
Hash	Hash(netid) mod 3 = 0	Hash(netid) mod 3 = 1	Hash(netid) mod 3 = 2
All / Broadcast			



Data Distribution Strategies



- Example: distribute data across three nodes

	Node 1	Node 2	Node 3
Range	Netid in [aa0000, hz9999]	Netid in [ia0000, sz0000]	Netid in [ta0000, zz9999]
Round robin	Row 1, row 4, row 7, ...	Row 2, row 5, row 8, ...	Row 3, row 6, row 9, ...
Hash	Hash(netid) mod 3 = 0	Hash(netid) mod 3 = 1	Hash(netid) mod 3 = 2
All / Broadcast	All	All	All



Data Distribution Strategies



	Range	Round Robin	Hash	All / Broadcast
Need to select a column?	Yes	No	Yes	No
Load balance?	Hard	Easy	Easy	N/A



Data Distribution Strategies



	Range	Round Robin	Hash	All / Broadcast
Need to select a column?	Yes	No	Yes	No
Load balance?	Hard	Easy	Easy	N/A
Impact on joins	Can be useful but makes load balancing worse	Useless	Good for large tables	Good for small tables



Distribution Strategies and Joins



netid	first_name	last_name	major
as1234	Alice	Smith	CSE
bw5678	Bob	Williams	ECE

major	enrollment
CSE	400
ECE	300

Round robin distribution

Joined records are not co-located on the same node -> network comm. overhead

netid	f_name	l_name	major
as1234	Alice	Smith	CSE
bw5678	Bob	Williams	ECE
major	enrollment	Node 1	
ECE	300		

netid	f_name	l_name	major
as1234	Alice	Smith	CSE
bw5678	Bob	Williams	ECE
major	enrollment	Node 2	
CSE	400		

Distribution Strategies and Joins

netid	first_name	last_name	major
as1234	Alice	Smith	CSE
bw5678	Bob	Williams	ECE

major	enrollment
CSE	400
ECE	300

Hash distribution (on “major” table column)

Joined records are co-located on the same node -> No network comm. overhead

netid	f_name	l_name	major
as1234	Alice	Smith	CSE
bw5678	Bob	Williams	ECE
major	enrollment	Node 1	
CSE	400		

netid	f_name	l_name	major
as1234	Alice	Smith	CSE
bw5678	Bob	Williams	ECE
major	enrollment	Node 2	
ECE	300		

Distribution Strategies and Joins

netid	first_name	major	grad_yr
as1234	Alice	CSE	2027
bw5678	Bob	ECE	2026

major	enrollment
CSE	400
ECE	300

grad_yr	enrollment
2026	1500
2027	1600

netid	f_name	major	grad_yr
as1234	Alice	CSE	2027
major	enr.	Node 1	
CSE	400		

netid	f_name	major	grad_yr
bw5678	Bob	ECE	2026
major	enr.	Node 2	
ECE	300		

Distribution Strategies and Joins

netid	first_name	major	grad_yr
as1234	Alice	CSE	2027
bw5678	Bob	ECE	2026

major	enrollment
CSE	400
ECE	300

grad_yr	enrollment
2026	1500
2027	1600

All / broadcast distribution for small tables

Data is duplicated but network communication is minimized

netid	f_name	major	grad_yr
as1234	Alice	CSE	2027
major	enr.	grad_yr	enr.
CSE	400	2026	1500
		2027	1600

netid	f_name	major	grad_yr
bw5678	Bob	ECE	2026
major	enr.	grad_yr	enr.
ECE	300	2026	1500
		2027	1600



© 2026 James Jianqiao Yu
Harbin Institute of Technology, Shenzhen

All rights reserved.

These lecture materials are intended solely for educational use in the course *Distributed Systems*. Permission is granted to students enrolled in the course to reproduce these materials, in whole or in part, for personal, non-commercial use only (including printing and photocopying). Any further reproduction or distribution, including but not limited to public posting on websites, social media, forums, or file-sharing platforms, distribution to non-enrolled individuals, or any commercial use, is prohibited without the prior written permission of the author.

This slide deck is adapted from Princeton University's *Distributed Systems* course materials, originally created by Michael Freedman, Kyle Jamieson, and Wyatt Lloyd, and licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported (CC BY-NC-SA 3.0) License. The original authors and source are gratefully acknowledged.